

Statistical Learning and Data Analysis



UNIVERSITÀ DEGLI STUDI DI NAPOLI

FEDERICO II

Data Science

Prof. Roberta Siciliano

Dataset : [Cryptocurrency Price Analysis Dataset](#)

Project Code: https://colab.research.google.com/drive/1VUYCAbnx8AE-rIN6ZgPZyAUmkPw_hHXy?usp=sharing

Presented by:

Sonal Sunit Deshmukh (D3000157)

1. Problem Statement	4
1.1. What is Contents	
Cryptocurrency?.....	4
1.2. How Cryptocurrencies Work:.....	5
1.3. Why Should We Analyze Cryptocurrency Prices?	5
1.4. Objectives:.....	7
2. Libraries.....	8
3. Exploratory Data Analysis	9
3.1. Data Understanding	9
3.2. Data Visualization.....	10
• Histogram.....	10
• Heatmap:.....	12
• BarPlot.....	13
• Scatter plot.....	14
• Pair-plot.....	16
3.3. Missing Values.....	18
3.4. Data Integration	19
4. Data Preprocessing	21
4.1. Handling Outliers	21
4.2. Feature Engineering.....	22
4.3. Training and testing of data	28
4.4. Data Balancing	30
4.5. Under-sampling.....	32
4.6. Hypothesis Testing	34
• Hypothesis 1.....	35
• Hypothesis 2.....	35
5. Model Training	37
5.1. Logistic Regression	38
5.2. Decision Tree	39

5.3.	Random Forest	40
6.	Confusion Matrix.....	42
6.1.	Confusion Matrix for Logistic Regression.....	42
6.2.	Confusion Matrix for Decision Tree	43
6.3.	Confusion Matrix for Random Forest.....	44
6.4.	Model Performance	45
7.	Conclusion.....	48
8.	Key Takeaways from the Project	49
8.1.	Can we predict future cryptocurrency prices using historical data?	49
8.2.	What is the impact of market capitalization on cryptocurrency prices?	49
8.3.	How do balancing techniques affect model performance in classification tasks?	49
8.4.	How Our Model Helps in Crypto Analysis	50
9.	Future Use of Classification Models in Crypto Analysis	51
10.	Future Research Direction.....	54

1. Problem Statement

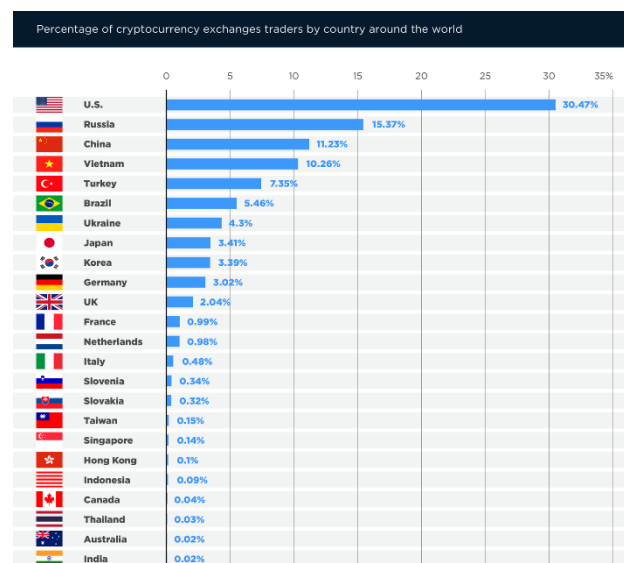
The **Cryptocurrency Price Analysis Dataset** contains historical price data for multiple cryptocurrencies. The goal of this analysis is to develop predictive models to estimate future cryptocurrency prices based on historical trends and market indicators. The "Cryptocurrency Price Analysis Dataset: BTC, ETH, XRP, LTC (2018-2023)" is a comprehensive dataset that captures the daily price movements of six popular cryptocurrencies. It covers a period from January 1, 2018, to May 31, 2023, providing a valuable resource for researchers, analysts, and enthusiasts interested in studying the historical price behavior of these digital assets.

1.1. What is Cryptocurrency?

A **cryptocurrency** is a **digital or virtual currency** that uses **cryptography** for security and operates on **decentralized** blockchain technology. Unlike traditional currencies issued by governments (**fiat currencies** like USD, EUR, or INR), cryptocurrencies are not controlled by any central authority, such as a bank or government.

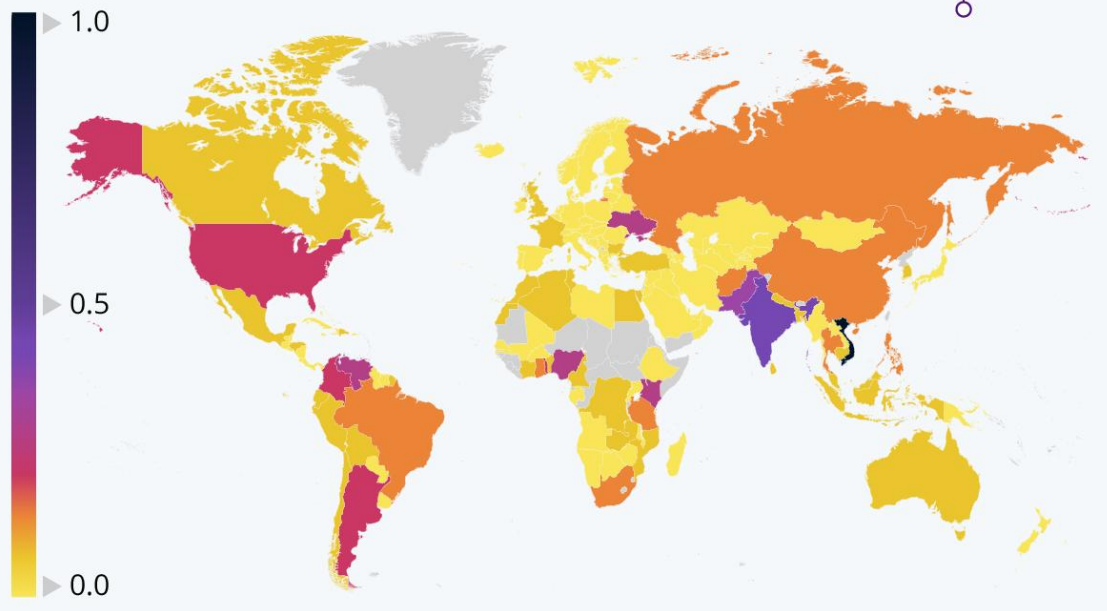
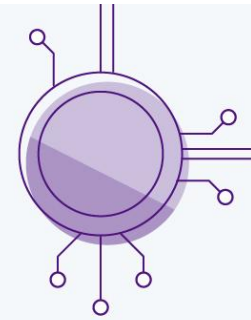
Popular Cryptocurrencies:

- **Bitcoin (BTC)** – The first and most well-known cryptocurrency, created in 2009 by an unknown person or group using the name **Satoshi Nakamoto**.
- **Ethereum (ETH)** – A platform that allows developers to build decentralized applications (**DApps**) and smart contracts.
- **Binance Coin (BNB)** – Used on the Binance exchange for transactions and trading fees.
- **Ripple (XRP)** – Focused on fast, low-cost international payments.



Where Cryptocurrency Is Most Heavily Used

Index value of global cryptocurrency adoption*



1.2. How Cryptocurrencies Work:

- **Transactions:** When a user sends cryptocurrency, the transaction is recorded on the blockchain.
- **Verification:** A network of computers (**nodes** or **miners**) verifies transactions through a process like **Proof of Work (PoW)** or **Proof of Stake (PoS)**.
- **Mining/Staking:** New coins are generated through mining (solving complex algorithms) or staking (holding coins to support the network).
- **Storage:** Cryptocurrencies are stored in **digital wallets** (software or hardware-based).

1.3. Why Should We Analyze Cryptocurrency Prices?

Analyzing cryptocurrency prices is crucial due to the volatile nature of the crypto market and its growing impact on global finance. Here are some key reasons why cryptocurrency price analysis is important:

1. Understanding Market Trends

Cryptocurrencies exhibit high price fluctuations, and analyzing historical data helps identify **trends, cycles, and patterns** that can aid investors in making informed decisions.

2. Predicting Future Prices

By using machine learning models like **Linear Regression, Decision Tree, and Random Forest**, we can attempt to predict future price movements based on past data.

These predictions can help traders **optimize buy and sell strategies** to maximize profits and minimize losses.

3. Risk Management and Volatility Analysis

Crypto markets are highly volatile, making risk management essential.

Statistical analysis helps in understanding **factors affecting price fluctuations**, reducing uncertainty for traders and investors.

4. Identifying Market Influences

Factors such as **news events, regulations, social media trends, and trading volumes** impact cryptocurrency prices.

Analyzing correlations between these factors and price changes helps investors **react proactively** to market movements.

5. Enhancing Trading Strategies

Technical analysis: Identifying patterns like moving averages, support/resistance levels, and RSI indicators.

Algorithmic trading: Using data-driven models to automate trading strategies for better efficiency.

6. Portfolio Optimization

Investors holding multiple cryptocurrencies can use data analysis to **diversify their portfolio** by selecting assets with lower risk and higher return potential.

7. Fraud and Anomaly Detection

Crypto markets are prone to **manipulation, pump-and-dump schemes, and wash trading**.

Statistical models can help detect anomalies, ensuring safer investment opportunities.

8. Understanding the Impact of External Factors

Cryptocurrency prices can be influenced by global events, **regulatory changes, economic crises, and technological advancements.**

Analyzing these relationships helps predict how such events might impact the market.

1.4. Objectives:

- Understand the distribution and trends of cryptocurrency prices over time.
- Perform correlation analysis to detect relationships between different cryptocurrencies and market variables.
- Test whether cryptocurrency price changes follow a normal distribution.
- Perform hypothesis testing on the relationship between price and trading volume. Analyze whether market capitalization significantly affects price stability.
- Use statistical tests to validate assumptions about cryptocurrency market behavior.
- Build regression models (Linear Regression, Ridge, Lasso) to predict future cryptocurrency prices Or Build classification models (Logistic Regression, Decision Trees, Random Forest) to categorize price trends.
- Identify trading patterns and market trends that can inform investment decisions.
- Offer insights on risk management based on volatility and market movements.
- Discuss how data balancing and feature engineering impact model accuracy.

2. Libraries

Pandas: It is used for handling and analyzing structured data (tables, CSV, Excel, SQL). It performs operations like filtering, grouping, merging, and aggregation. And handle missing values and convert data types

Numpy: It provides support for handling large arrays and mathematical operations. It efficiently stores and manipulates large datasets as well as supports linear algebra and statistical functions.

Matplotlib: It creates basic charts like line plots, histograms, scatter plots. It provides full control over figure properties and customizable labels, titles, and legends.

Seaborn: It is used for more visually appealing statistical plots and simplifies complex visualizations. It supports correlation heatmaps, box plots, violin plots, etc.

sklearn.model_selection: It splits dataset into training and testing sets; performs cross-validation. It ensures better model performance evaluation and avoids overfitting and underfitting.

sklearn.linear_model: It implements various regression techniques.

Ridge() – L2 regularization to prevent overfitting.

Lasso() – L1 regularization (feature selection).

LogisticRegression() – Classification model.

sklearn.tree: It is Used for classification and regression tasks. It captures non-linear relationships and is prone to overfitting unless pruned.

sklearn.ensemble: It combines multiple models to improve performance.

RandomForestClassifier() – It uses multiple decision trees.

sklearn.metrics: It measures the accuracy of machine learning models.

mean_squared_error() – Regression error.

r2_score() – How well the model fits the data.

confusion_matrix() – Performance of classification models.

classification_report() – Precision, recall, F1-score.

sklearn.preprocessing: It converts raw data into a format suitable for ML models.

imblearn.over_sampling: It increases samples of minority class in imbalanced datasets.

imblearn.under_sampling: It reduces majority class samples in imbalanced datasets.

3. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is the process of **analyzing, summarizing, and visualizing** a dataset to understand its structure, patterns, and key insights before applying machine learning models.

Why is EDA Important?

- Helps in detecting **missing values, outliers, and data inconsistencies**.
- Identifies **patterns, trends, and relationships** between variables.
- Provides insights for **feature selection and engineering**.
- Assists in choosing the right **machine learning model**.

3.1. Data Understanding

The dataset contains **7,899 rows** and **6 columns**.

Date: The date of the recorded cryptocurrency data.

Cryptocurrency Name: The name of the cryptocurrency (e.g., Bitcoin, Ethereum, etc.).

Open Price: The price of the cryptocurrency at the start of the trading period.

Close Price: The price of the cryptocurrency at the end of the trading period.

High Price: The highest price reached by the cryptocurrency during the trading period.

Low Price: The lowest price reached by the cryptocurrency during the trading period.

Data types:

- **4 columns** are of type **float64** (numerical price data).
- **2 columns** are of type **object** (text-based data: Crypto name and Date).

We used following function to describe our data set.

`df.info()` provides a summary of a pandas DataFrame.

`df.describe()` provides statistical summary of numerical columns and includes count, mean, standard deviation, min, max, and quartiles (25%, 50%, 75%). It helps understand distribution, trends, and outliers in numerical data.

`df.columns` returns a list of column names and is useful to check if column names need renaming. It helps in selecting or renaming specific columns.

df.head() gives a quick snapshot of the dataset structure by displaying the first n rows of dataset.

df.tail() helps check the latest data entries by displaying last n rows.

df.shape returns the dimensions of the DataFrame in the format (rows, columns).

```
# Get summary statistics of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7899 entries, 0 to 7898
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   Crypto  7899 non-null   object
 1   Date    7899 non-null   object
 2   Open    7899 non-null   float64
 3   High    7899 non-null   float64
 4   Low     7899 non-null   float64
 5   Close   7899 non-null   float64
dtypes: float64(4), object(2)
memory usage: 370.4+ KB
```

```
df.describe()
```

	Open	High	Low	Close
count	7899.000000	7899.000000	7899.000000	7899.000000
mean	5479.620176	5623.947143	5324.149024	5480.870398
std	12010.579332	12325.039614	11665.236339	12011.567525
min	0.140000	0.150000	0.120000	0.140000
25%	12.720000	13.410000	12.405000	12.680000
50%	170.910000	176.240000	163.320000	170.890000
75%	3879.000000	3962.000000	3757.000000	3878.935000
max	67802.000000	68925.000000	66112.000000	67802.000000

```
# Display the first few rows of the dataset
df.head()
```

	Crypto	Date	Open	High	Low	Close
0	BTC	12/31/19	7254.0	7309.0	7132.0	7171.0
1	BTC	12/30/19	7402.0	7430.0	7217.0	7254.0
2	BTC	12/29/19	7334.0	7529.0	7295.0	7402.0
3	BTC	12/28/19	7235.0	7359.0	7235.0	7334.0
4	BTC	12/27/19	7208.0	7267.0	7087.0	7235.0

```
# Display the last few rows of the dataset
df.tail()
```

	Crypto	Date	Open	High	Low	Close
7894	XRP	1/5/19	0.36	0.36	0.35	0.36
7895	XRP	1/4/19	0.36	0.36	0.35	0.36
7896	XRP	1/3/19	0.37	0.38	0.35	0.36
7897	XRP	1/2/19	0.36	0.37	0.36	0.37
7898	XRP	1/1/19	0.35	0.36	0.34	0.36

```
df.shape
```

```
(7899, 6)
```

3.2. Data Visualization

Data visualization refers to the graphical representation of data to help understand its underlying patterns, trends, and relationships. It involves creating charts, graphs, and plots that make it easier to identify key insights, detect anomalies, and summarize the distribution and relationships in the data.

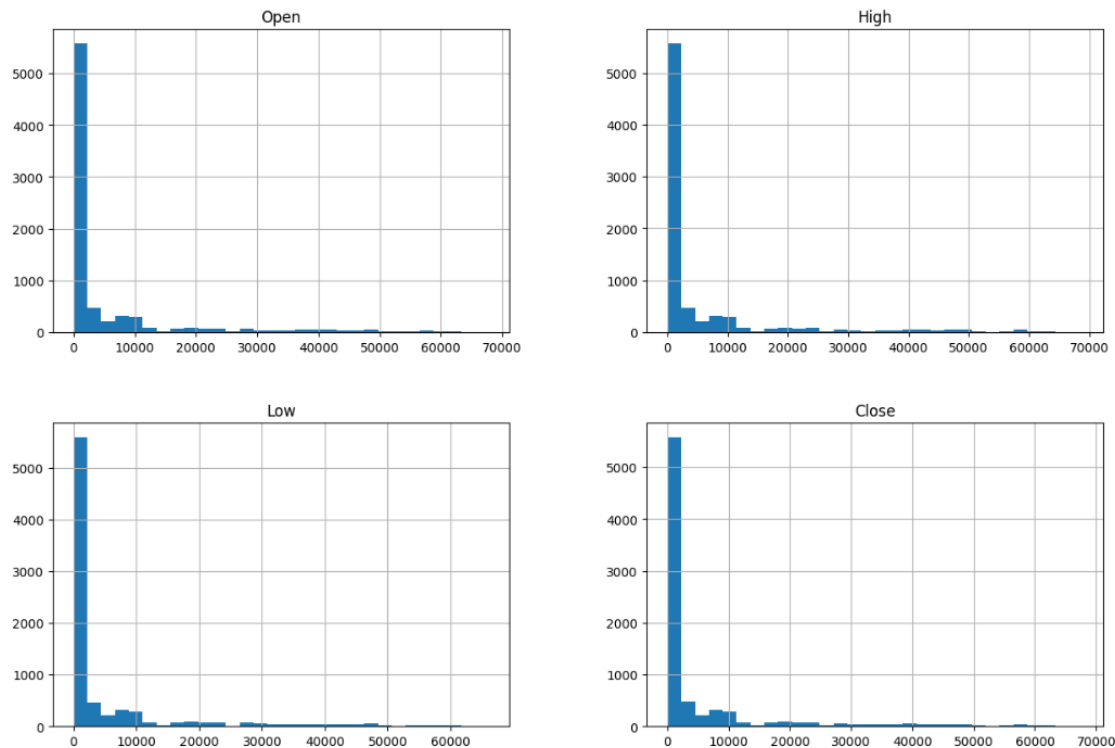
- [Histogram:](#)

A histogram represents the distribution of a single numerical variable by dividing the entire range of values into bins and counting the number of values that fall into each bin.

It helps understand the frequency distribution, such as whether data is skewed, normally distributed, or has outliers.

```
# Histogram of numerical features
df[numeric_cols].hist(bins=30, figsize=(15, 10))
plt.suptitle('Histograms of Numerical Features')
plt.show()
```

Histograms of Numerical Features



Right-Skewed Distribution:

All four histograms (Open, High, Low, and Close) exhibit a highly right-skewed distribution. Most values are concentrated on the lower end, with a small number of very high values.

Presence of Outliers:

There are a few data points in the high-price range, indicating extreme values or outliers in the dataset. These high prices may belong to expensive cryptocurrencies like Bitcoin or Ethereum.

Majority of Cryptocurrencies Have Low Prices:

The highest frequency of values appears in the lower price ranges (between 0 and 10,000). This suggests that most cryptocurrencies in the dataset have a relatively low price, while only a few have exceptionally high prices.

Consistency Across Open, High, Low, and Close Prices:

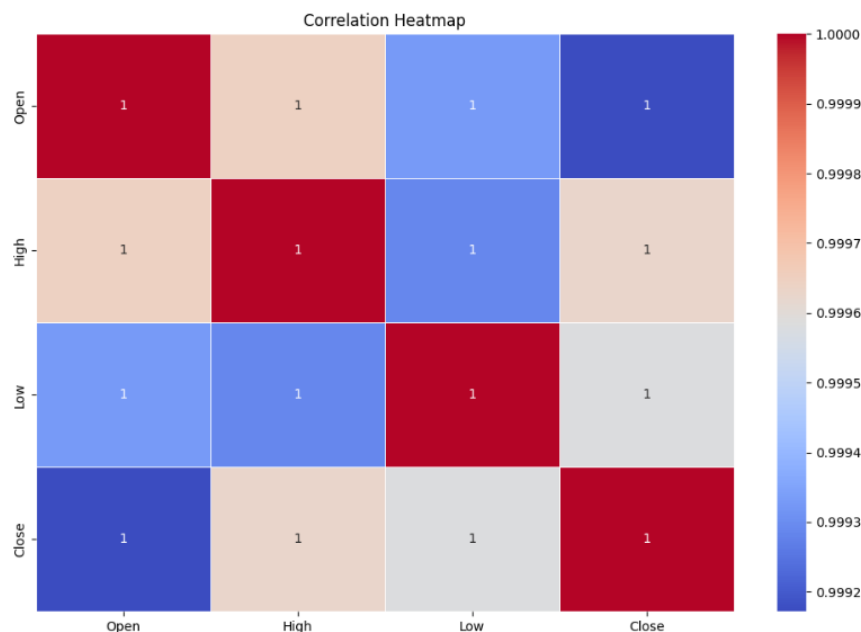
Since all four features (Open, High, Low, and Close) have similar distributions, it indicates price fluctuations remain proportional across different price points.

- [Heatmap:](#)

A heatmap is a visualization that represents data using colors to indicate different values. It is commonly used to visualize correlation matrices to understand relationships between numerical variables.

It helps in identifying correlations between different cryptocurrency prices.

```
# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.select_dtypes(include=['int64', 'float64']).corr(),
            annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```



Strong Positive Correlation (≈ 1.0) Between All Features: The heatmap shows that Open, High, Low, and Close prices have nearly perfect correlations (~ 1.0) with each other. This suggests that these features move together in a synchronized manner. Such high correlation is expected in financial time series, as Open, High, Low, and Close prices are closely related.

Redundant Features for Modeling: Since all features are highly correlated, including all of them in a machine learning model might introduce multicollinearity. Dimensionality reduction

techniques (like PCA) or selecting just one of these features (e.g., Close price) might be beneficial for predictive modeling.

Price Movements are Proportional Across Cryptocurrencies: The similar correlation structure suggests that regardless of the cryptocurrency, price changes are proportional across Open, High, Low, and Close values.

- [BarPlot](#)

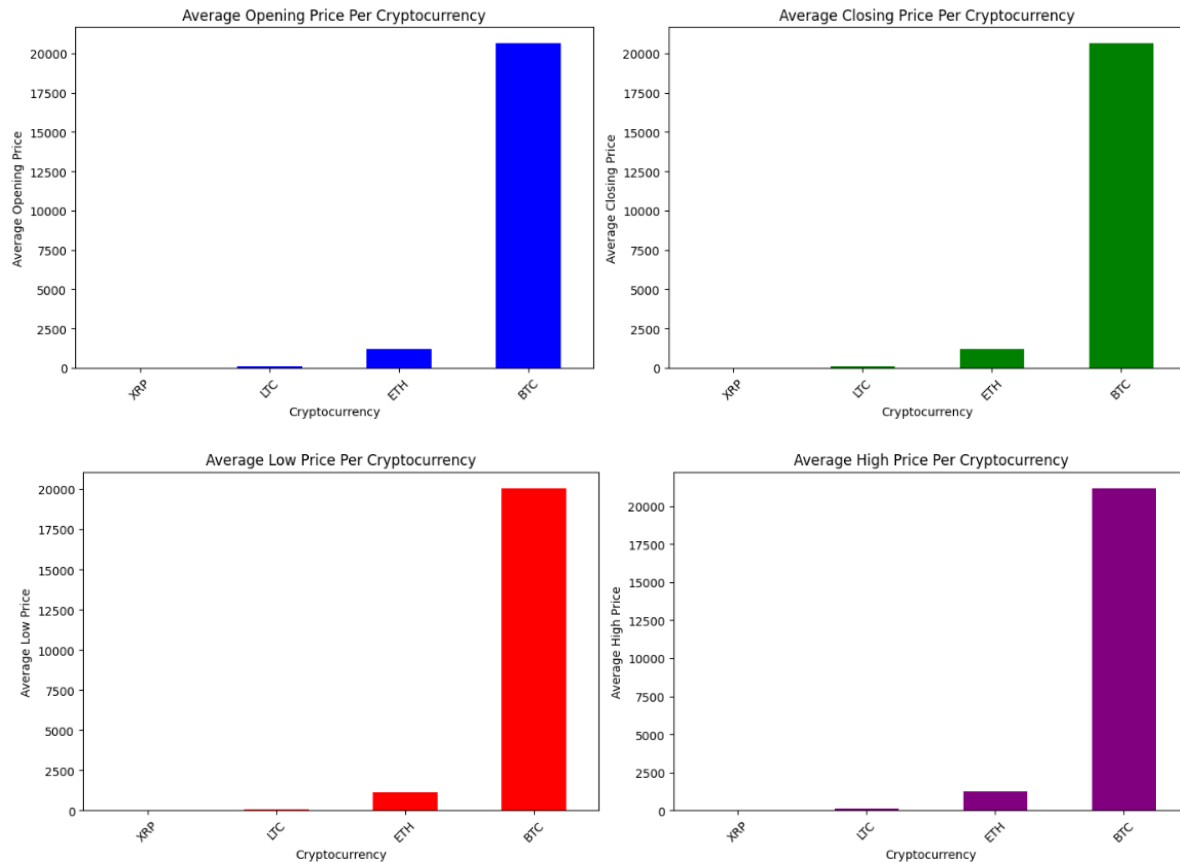
```
# Bar Plot for Average opening, closing, high and low Price Per
Cryptocurrency

# Define price types and colors
price_types = {'Open': 'blue', 'Close': 'green', 'Low': 'red', 'High':
'purple'}
titles = {'Open': 'Opening', 'Close': 'Closing', 'Low': 'Low', 'High':
'High'}

# Create subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, (price_type, color) in enumerate(price_types.items()):
    df.groupby('Crypto')[price_type].mean().sort_values().plot(kind='bar',
color=color, ax=axes[i])
    axes[i].set_title(f'Average {titles[price_type]} Price Per
Cryptocurrency')
    axes[i].set_xlabel('Cryptocurrency')
    axes[i].set_ylabel(f'Average {titles[price_type]} Price')
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



From above bar graphs, we can observe that:

BTC (Bitcoin) has the highest prices across all four charts.

Other cryptocurrencies (XRP, LTC, ETH) have significantly lower average prices.

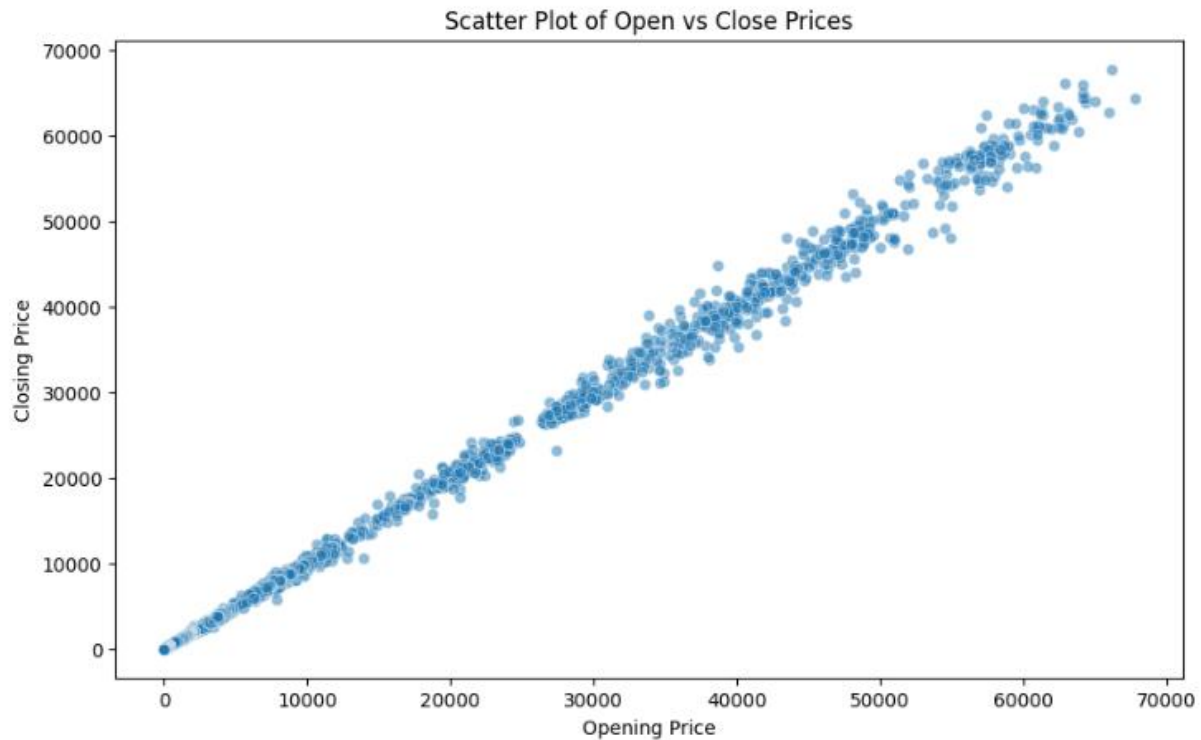
The values for opening, closing, high, and low prices are almost the same, indicating **less variation** in the daily price movements.

The dataset likely contains **highly imbalanced price distributions**, as BTC dominates in price compared to the others.

- [Scatter plot](#)

A scatter plot represents two numerical variables on an X-Y axis to observe relationships or patterns.

It is used to check for correlation between variables (positive, negative, or no correlation).



(Opening Price vs. Closing Price):

Strong Linear Relationship: The scatter plot shows a strong positive linear correlation between the Opening Price and Closing Price of cryptocurrencies. This suggests that the price at which a cryptocurrency opens is a strong predictor of its closing price for the same period.

Minimal Price Fluctuation: The data points are closely clustered around a straight diagonal line, indicating that the opening and closing prices are very similar. This suggests that cryptocurrency prices do not fluctuate drastically within the same period for most instances.

Possible Market Stability: The consistency in opening and closing prices indicates a relatively stable market trend, at least for the period covered in the dataset. Large deviations (outliers) could signify volatile trading days, but they appear minimal.

Predictability in Price Movements: Since opening prices are highly correlated with closing prices, simple regression models could effectively predict the closing price based on the opening price. More advanced techniques could be explored if needed for intraday volatility analysis.



(Low Price vs. High Price):

Strong Positive Correlation: The scatter plot shows a strong linear relationship between Low Price and High Price of cryptocurrencies. This means that when the low price of a cryptocurrency increases, its high price also increases proportionally.

Small Price Fluctuations: Most data points lie close to the diagonal, indicating that the difference between low and high prices is relatively small for most cryptocurrencies. This suggests that daily or periodic price swings are not extreme in most cases.

Some Outliers Exist: A few points deviate slightly from the diagonal, which could indicate higher volatility in those specific cases. These may correspond to highly volatile cryptocurrencies or sudden price spikes/drops due to market events.

Price Stability Across Cryptocurrencies: Since the relationship between low and high prices is almost perfectly linear, it indicates consistent price behavior across different cryptocurrencies in the dataset.

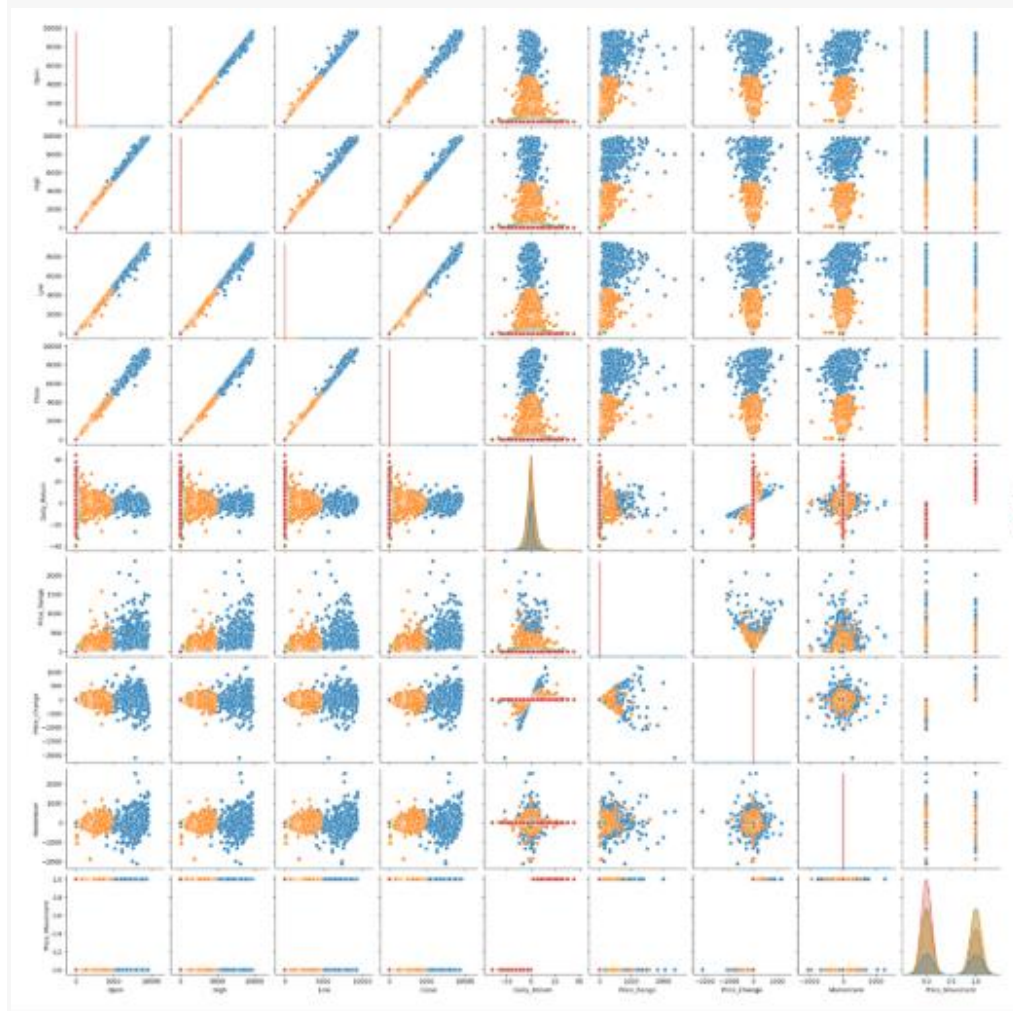
- [Pair-plot](#)

A pairplot (from Seaborn library) creates scatter plots between all numerical feature pairs in a dataset.

It is useful for visualizing the relationships between multiple numerical variables and detecting trends or correlations.

Tightly clustered points along a diagonal line → Strong correlation. Randomly scattered points → Weak or no correlation. Separated clusters (with hue) → Different groups of cryptocurrencies behave differently. Spread-out distributions → High volatility or diverse price ranges.

```
# Pairplot for all numerical features
sns.pairplot(df, hue='Crypto')
plt.show()
```



From this pair-plot we know that,

Highly Correlated Features: Features like opening, closing, high, and low prices are likely highly correlated, making some of them redundant for modeling.

Outliers: The presence of **some extreme points** suggests possible outliers (e.g., BTC price spikes).

Feature Scaling Needed: Since BTC prices are much higher than others, normalization or log transformation might be needed for better modeling.

3.3. Missing Values

Missing values occur when data is unavailable or not recorded in a dataset. They can significantly impact machine learning models.

Types of Missing Values:

1. **MCAR (Missing Completely at Random)** – No pattern; occurs randomly.
2. **MAR (Missing at Random)** – Missing values depend on some known variable.
3. **MNAR (Missing Not at Random)** – Missing values depend on the missing variable itself.

How to Handle Missing Values?

1. **Removing Missing Values** – Use `dropna()`, but this may lead to data loss.
2. **Imputation (Filling Missing Values)** – Replace missing values with:
 - Mean (`df.fillna(df.mean())`)
 - Median (`df.fillna(df.median())`)
 - Mode (`df.fillna(df.mode().iloc[0])`)
 - Forward Fill / Backward Fill (`df.fillna(method='ffill')` or `df.fillna(method='bfill')`)
3. **Predictive Imputation** – Using machine learning models to estimate missing values.

```
# Handling Missing Values
missing_values = df.isnull().sum()
print("Missing Values Before Handling:")
print(missing_values)

if missing_values.sum() == 0:
    print("\n No missing values detected. Proceeding with outlier detection
    and feature engineering.\n")
else:
    # Fill missing values with median for numerical columns
    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
    print("\n Missing Values After Handling:")
    print(df.isnull().sum())
```

```
Missing Values Before Handling:
```

```
Crypto    0
Date      0
Open      0
High      0
Low       0
Close     0
dtype: int64
```

```
No missing values detected. Proceeding with outlier detection and feature engineering.
```

3.4. Data Integration

Data Integration is the process of combining data from multiple sources into a unified dataset. It ensures consistency, completeness, and accuracy in data analysis.

Why is Data Integration Important?

- Helps in merging data from different sources (e.g., databases, APIs, CSV files).
- Reduces redundancy and inconsistency.
- Improves data quality for machine learning and analytics.

```
# Data Integration
# Assuming integration involves aggregating data by date across all
cryptocurrencies
# Calculate daily mean prices and total volume
daily_data = df.groupby('Date').agg({
    'Open': 'mean',
    'Close': 'mean',
    'High': 'mean',
    'Low': 'mean'
}).reset_index()

# Display the aggregated daily_data DataFrame
print("Aggregated Daily Data:")
print(daily_data.head())
```

```
Aggregated Daily Data:
```

	Date	Open	Close	High	Low
0	2018-01-01	3743.1750	3629.9350	3761.9625	3451.1975
1	2018-01-02	3630.1025	3972.7025	4091.0300	3484.9875
2	2018-01-03	3972.1975	4060.1950	4151.5775	3920.6750
3	2018-01-04	4059.9925	4026.4100	4158.5650	3839.3375
4	2018-01-05	4026.4075	4507.2750	4545.4600	3988.7875

The dataset is grouped by 'Date', and the mean values for 'Open', 'Close', 'High', and 'Low' prices are calculated. The output table shows the first five rows of aggregated daily cryptocurrency prices.

Trend in Prices: The Open, Close, High, and Low prices fluctuate daily.

Data Integration Outcome: This process combines multiple cryptocurrency price data into a single daily record, making it easier to analyze overall market trends.

It is useful for time series analysis or trend forecasting in cryptocurrency prices.

4. Data Preprocessing

Data preprocessing involves preparing raw data for analysis by cleaning, transforming, and organizing it. This step is crucial to ensure the quality and accuracy of the data used in analysis.

Why we do data preprocessing?

- **Improving Model Performance:** Clean, well-preprocessed data leads to more accurate and reliable models.
- **Ensuring Generalization:** Proper data splitting and balancing ensure the model performs well on unseen data.
- **Reducing Bias:** Handling outliers and balancing data reduce biases that can skew results.

4.1. Handling Outliers

Outliers are data points that significantly differ from other observations. They can skew results and affect the performance of models. Handling outliers involves identifying and either removing or transforming them to minimize their impact.

```
# Outlier Detection & Removal

# Display the shape of the dataset before outlier removal
print("\nDataset shape before outlier removal:", df.shape)

Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1
outlier_condition = ((df[numeric_cols] < (Q1 - 1.5 * IQR)) |
(df[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)

print("\nNumber of outliers detected:", outlier_condition.sum())
df = df[~outlier_condition]
print("\nDataset shape after outlier removal:", df.shape)
```

```
Dataset shape before outlier removal: (7899, 6)
```

```
Number of outliers detected: 1233
```

```
Dataset shape after outlier removal: (6666, 6)
```

In our code, outliers are identified and removed using the Interquartile Range (IQR) method, a common statistical technique for detecting anomalies in data. Here's a breakdown of the process:

1. Calculate Quartiles and IQR:

First Quartile (Q1): The 25th percentile of the data.

Third Quartile (Q3): The 75th percentile.

Interquartile Range (IQR): The difference between Q3 and Q1, representing the middle 50% of the data.

2. Define Outlier Boundaries:

Lower Bound: $Q1 - 1.5 * IQR$

Upper Bound: $Q3 + 1.5 * IQR$

3. Identify and Remove Outliers:

Data points falling below the lower bound or above the upper bound are considered outliers.

These outliers are then removed from the dataset.

This method is effective for datasets with a roughly normal distribution. However, for skewed distributions or datasets with varying variability, alternative methods like the Z-score or domain-specific approaches might be more appropriate.

By applying the IQR method, we ensure that extreme values don't skew analysis, leading to more reliable and accurate results.

4.2. Feature Engineering

Feature creation involves generating new variables (features) from existing data to improve model performance. This can include combining features, creating interaction terms, or applying domain-specific transformations.

```
# Feature Engineering

# Ensure Date is in datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Daily Return Percentage
df['Daily_Return'] = ((df['Close'] - df['Open']) / df['Open']) * 100

# Price Range
```

```

df['Price_Range'] = df['High'] - df['Low']

#Price Change
df['Price_Change'] = df['Close'] - df['Open']

# Momentum (Close Price Difference over 3 days)
df['Momentum'] = df.groupby('Crypto')['Close'].transform(lambda x:
x.diff(periods=3))

# Price Movements
# 1 for upward movement, 0 for downward
df['Price_Movement'] = np.where(df['Daily_Return'] > 0, 1, 0)

# Print the first few rows with new features
print(df[['Crypto', 'Date', 'Daily_Return', 'Price_Range', 'Price_Change',
'Momentum', 'Price_Movement']].head())

```

	Crypto	Date	Daily_Return	Price_Range	Price_Change	Momentum \
0	BTC	2019-12-31	-1.144196	177.0	-83.0	NaN
1	BTC	2019-12-30	-1.999460	213.0	-148.0	NaN
2	BTC	2019-12-29	0.927188	234.0	68.0	NaN
3	BTC	2019-12-28	1.368348	124.0	99.0	163.0
4	BTC	2019-12-27	0.374584	180.0	27.0	-19.0

	Price_Movement
0	0
1	0
2	1
3	1
4	1

Daily Return Percentage: Measures the percentage change between the opening and closing price of the day. It can be useful for volatility and trend analysis.

Price Range: Calculates the difference between the highest and lowest price to assess daily volatility. It can be useful for capturing market volatility.

Price Change: Finds the absolute price movement between the opening and closing price. It helps understand the direction and magnitude of price movement.

Momentum: Tracks the closing price difference over 3 days to identify trends. It helps in identifying short-term trends.

Price Movement: A binary indicator (1 = Up, 0 = Down) showing if the price increased or decreased. It can be useful for classification models predicting market movement.

What is the outcome of feature engineering?

With the help of these new features, we **enhanced dataset** for better predictive power.

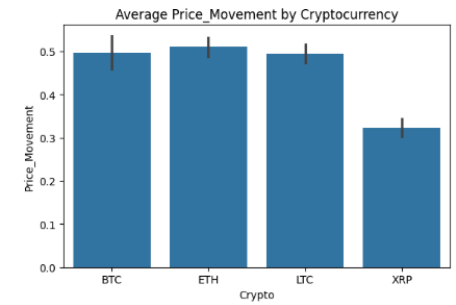
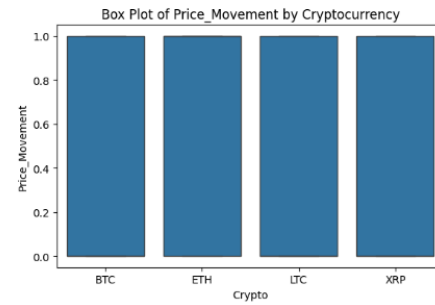
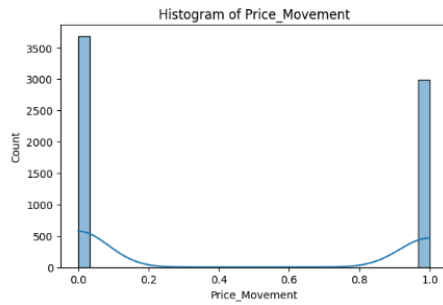
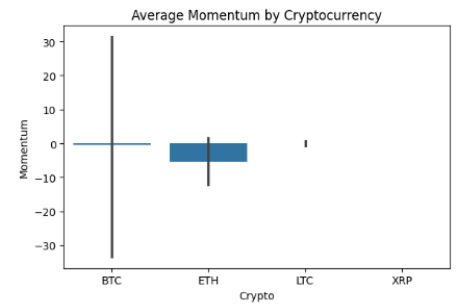
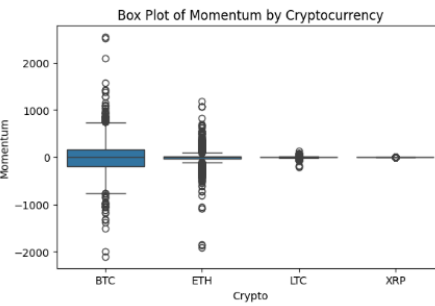
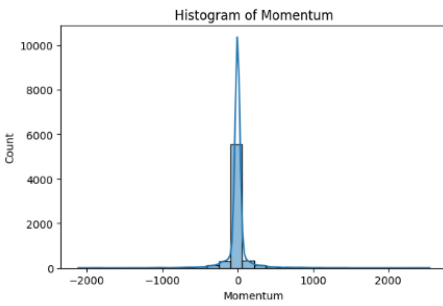
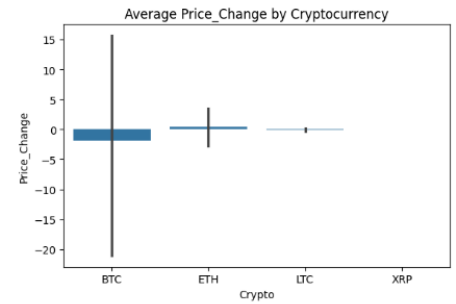
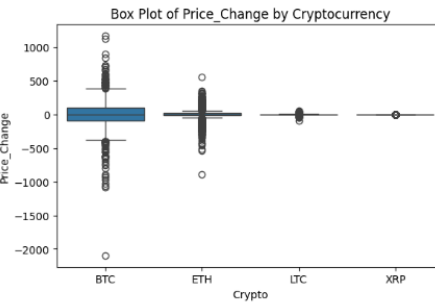
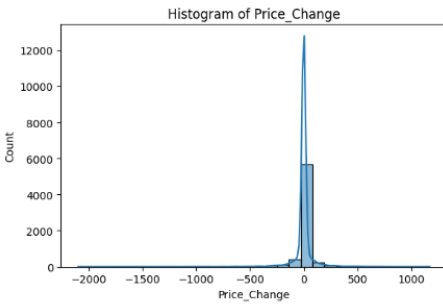
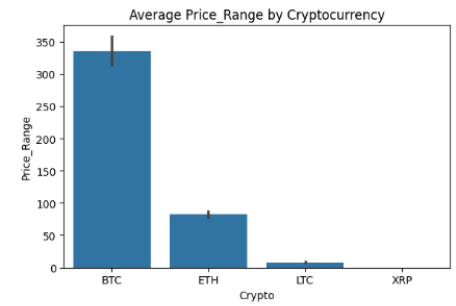
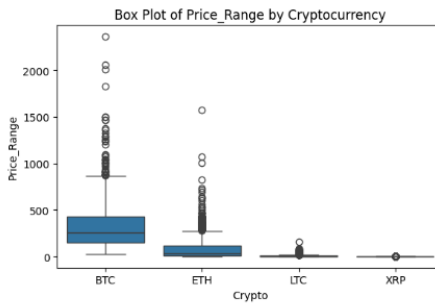
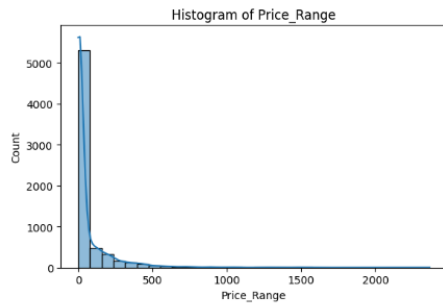
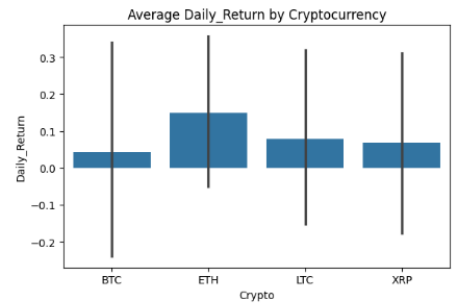
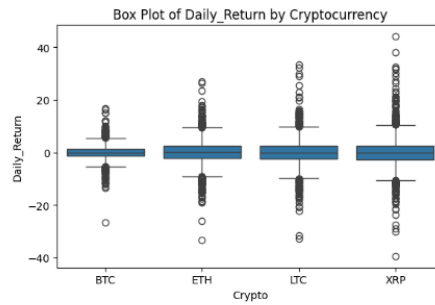
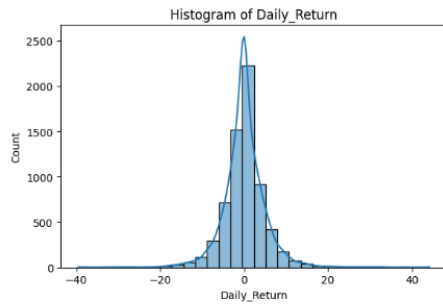
It **Improved market movement analysis** through price-based indicators and **Prepared data for machine learning models**, especially for trend analysis, regression, or classification tasks.

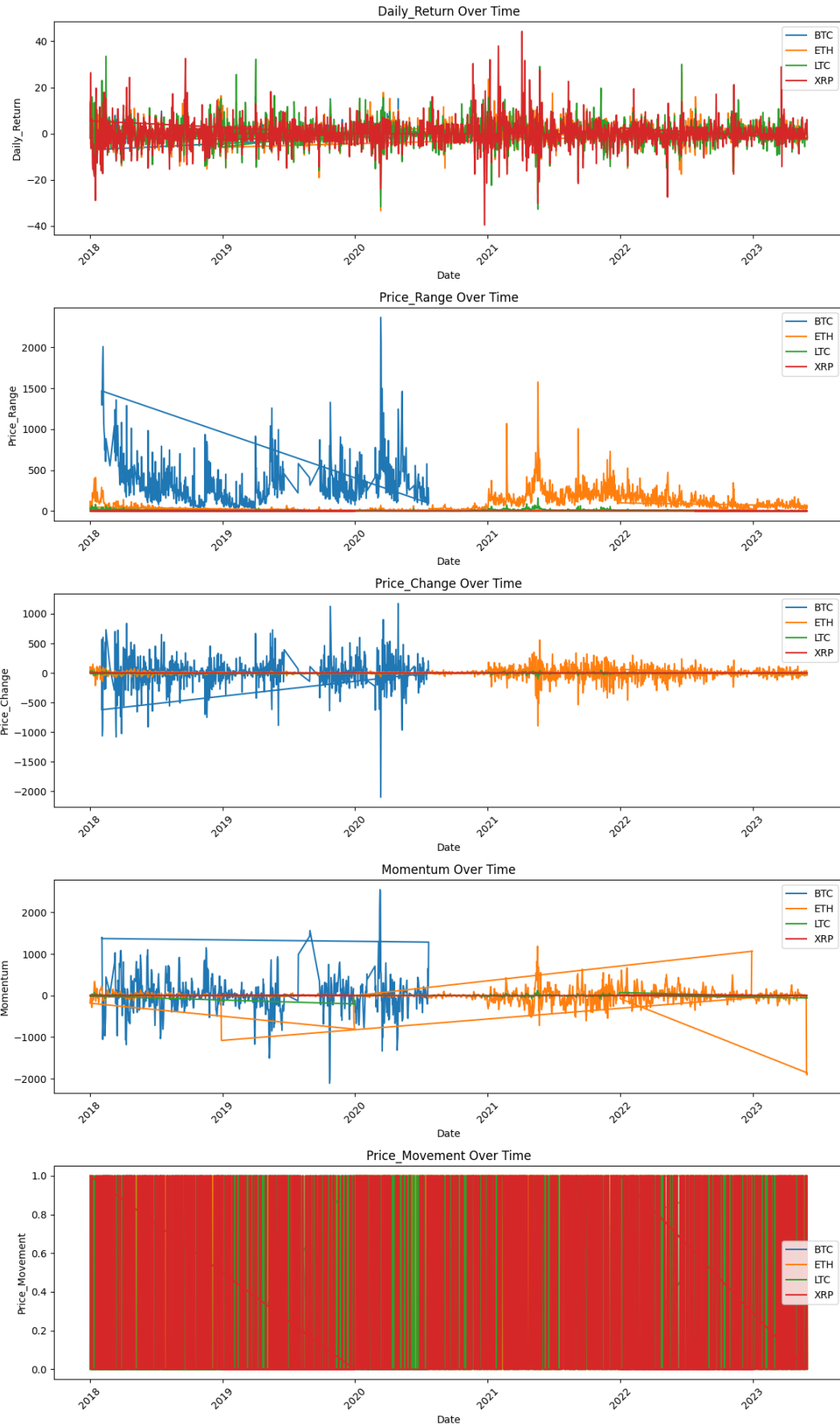
```
# Visualization of features

features = ['Daily_Return', 'Price_Range', 'Price_Change', 'Momentum',
            'Price_Movement']
fig, axes = plt.subplots(len(features), 3, figsize=(18, 20))

for i, feature in enumerate(features):
    # Histogram
    sns.histplot(df[feature], bins=30, kde=True, ax=axes[i, 0])
    axes[i, 0].set_title(f'Histogram of {feature}')
    # Box Plot
    sns.boxplot(x='Crypto', y=feature, data=df, ax=axes[i, 1])
    axes[i, 1].set_title(f'Box Plot of {feature} by Cryptocurrency')
    # Bar Graph
    sns.barplot(x='Crypto', y=feature, data=df, ax=axes[i, 2])
    axes[i, 2].set_title(f'Average {feature} by Cryptocurrency')

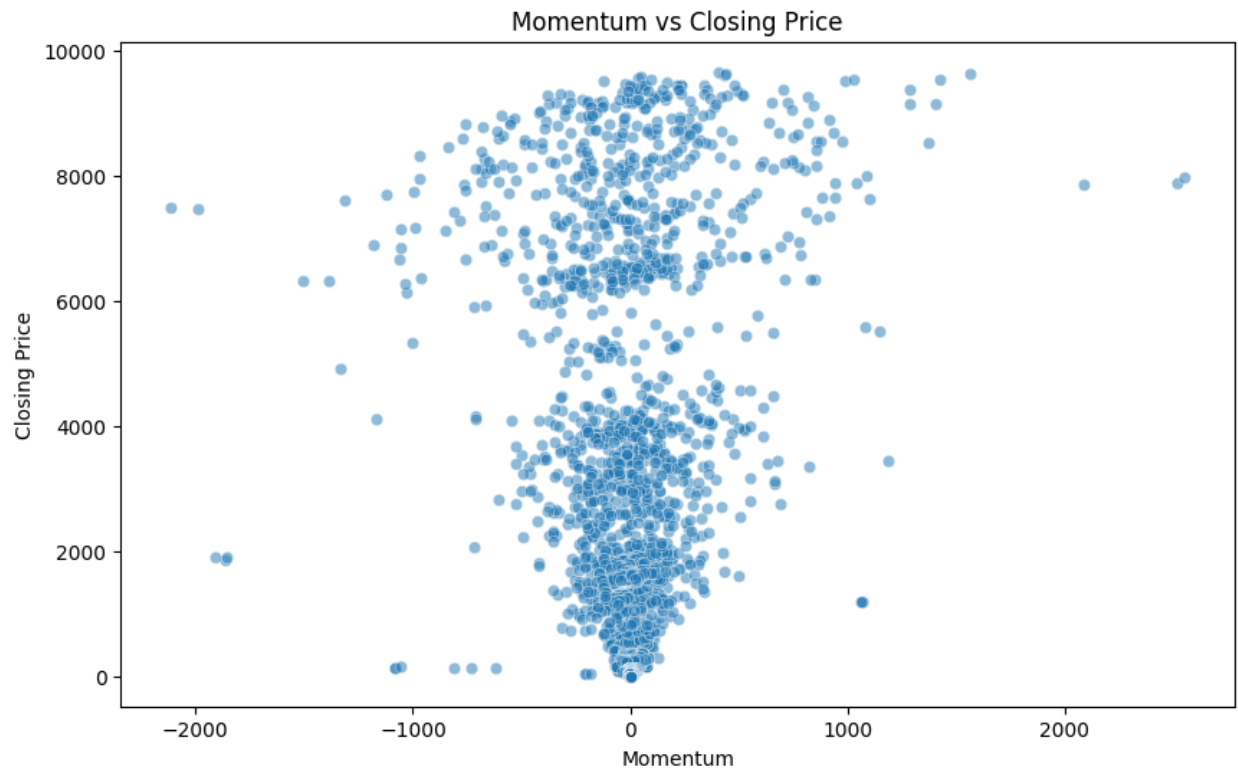
plt.tight_layout()
plt.show()
```



```
# Momentum Scatter Plot

plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['Momentum'], y=df['Close'], alpha=0.5)
plt.title('Momentum vs Closing Price')
plt.xlabel('Momentum')
plt.ylabel('Closing Price')
plt.show()
```



The scatter plot shows a wide dispersion of points, indicating that momentum (3-day price difference) does not have a strict linear relationship with the closing price.

Most data points are clustered around zero momentum, suggesting that short-term price changes are often small, with fewer extreme movements.

The presence of outliers on both positive and negative momentum sides suggests occasional high volatility and sudden price spikes or drops.

Higher closing prices seem to have a wider range of momentum values, meaning that as the price increases, the magnitude of fluctuations (both positive and negative) also increases.

Momentum provides insights into short-term price movements, but additional factors likely influence closing prices significantly.

4.3. Training and testing of data

Training data is used to build and train a model, while **testing data** is used to evaluate its performance. This separation helps ensure the model generalizes well to unseen data.

Why Do We Train and Test Data?

In **machine learning**, we train and test data to evaluate how well a model generalizes to new, unseen data.

- **Prevent Overfitting** – If we train on all the data, the model may memorize the dataset instead of learning general patterns.
- **Evaluate Model Performance** – The test data acts as unseen data to check if the model performs well beyond the training set.
- **Hyperparameter Tuning** – Helps in fine-tuning the model (e.g., adjusting regularization parameters).
- **Measuring Accuracy** – Enables checking metrics like accuracy, precision, recall, and F1-score.

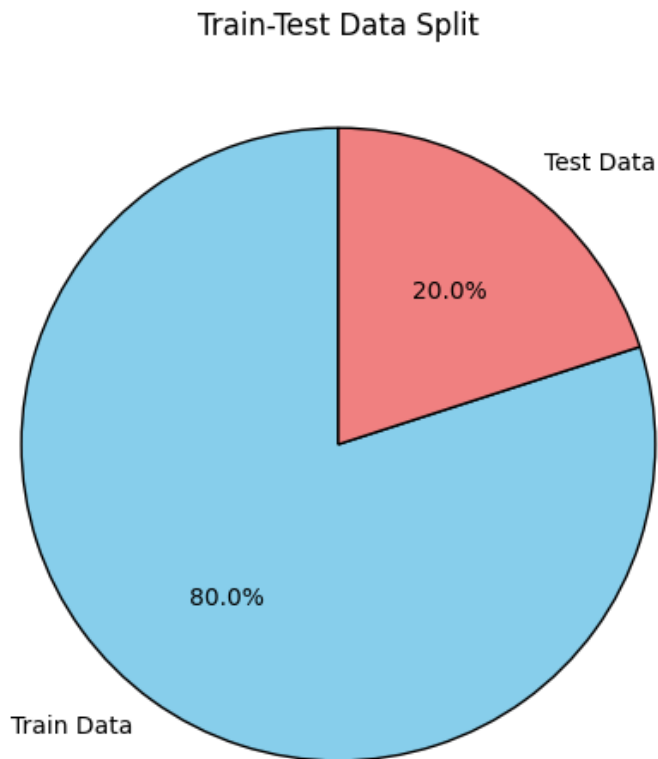
```
# Train-Test Split
X = df[numeric_cols].dropna()
y = np.where(df['Daily_Return'] > 0, 1, 0)
# Classification Target: 1 for upward movement, 0 for downward
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Count the number of samples in train and test sets
train_size = len(y_train)
test_size = len(y_test)

# Labels and sizes for the pie chart
labels = ['Train Data', 'Test Data']
sizes = [train_size, test_size]
colors = ['skyblue', 'lightcoral']

# Create a pie chart
plt.figure(figsize=(6, 6))
plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=colors,
startangle=90, wedgeprops={'edgecolor': 'black'})
plt.title('Train-Test Data Split')

plt.show()
```



```
X = df[numeric_cols].dropna()
```

- Selecting only **numeric features** and dropping missing values.

```
y = np.where(df['Daily_Return'] > 0, 1, 0)
```

- Converts the **Daily_Return** column into a **binary classification problem**:
 - 1 → If return is **positive** (price increased).
 - 0 → If return is **negative** (price decreased).

```
train_test_split(X, y, test_size=0.2, random_state=42)
```

- Splits X and y into **80% training data** and **20% test data**.
- `random_state=42` ensures **reproducibility** of the split.

Where,

`len(y_train)`: Number of training samples.

`len(y_test)`: Number of test samples. 80% Training Data & 20% Test Data

The pie chart visually confirms the split.

Helps understand how much data is used for training vs. evaluation.

4.4. Data Balancing

Data balancing addresses imbalances in the dataset, such as unequal class distributions. Techniques like **SMOTE** (Synthetic Minority Over-sampling Technique) and **undersampling** are used to balance the data.

Regarding dataset's outlier removal process, we've observed the following:

Removing outliers can significantly impact the data balancing process. Outliers often represent rare or extreme values, and their removal can alter the distribution of the dataset.

This change can affect the performance of models, especially in tasks like classification, where class balance is crucial. It's essential to carefully consider the implications of outlier removal and to explore alternative methods, such as robust statistical techniques, that can mitigate the influence of outliers without excluding them entirely.

What is SMOTE?

SMOTE (Synthetic Minority Over-sampling Technique) is a technique used to handle imbalanced datasets by generating synthetic samples for the minority class.

The **Synthetic Minority Over-Sampling Technique (SMOTE)** is a powerful method used to handle class imbalance in datasets. SMOTE handles this issue by generating samples of minority classes to make the class distribution balanced.

Why Do We Use SMOTE?

In classification problems, if one class has significantly fewer samples than the other(s), the model may become biased toward the majority class, leading to poor performance in predicting the minority class. SMOTE helps by:

- **Balancing the dataset** by creating synthetic data points for the minority class.
- **Preventing overfitting** caused by simply duplicating existing data points.
- **Improving model performance** by allowing better learning from both classes.

How Does SMOTE Work?

- a. **Selects a minority class sample** randomly.
- b. **Finds its k-nearest neighbors** (default is 5).

- c. **Generates synthetic data points** by interpolating between the sample and one of its neighbors.
- d. **Repeats the process** until the minority class is sufficiently balanced.

When to Use SMOTE?

When your dataset is highly **imbalanced** (e.g., 90% majority class, 10% minority class).

In **classification problems** (e.g., fraud detection, medical diagnoses).

Before training machine learning models to **ensure fair learning** across all classes.

```
# Data Balancing
# Print class distribution before balancing
print("\nClass distribution before balancing:")
print(pd.Series(y_train).value_counts())

# Visualize class distribution before balancing
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
pd.Series(y_train).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution Before Balancing")
plt.xlabel("Class")
plt.ylabel("Count")

# SMOTE Balancing
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

# Print class distribution after SMOTE
print("\nClass distribution after SMOTE:")
print(pd.Series(y_train_smote).value_counts())

# Visualize class distribution after SMOTE
plt.subplot(1, 2, 2)
pd.Series(y_train_smote).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution After SMOTE")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
Class distribution before balancing:
```

```
0    2958
```

```
1    2374
```

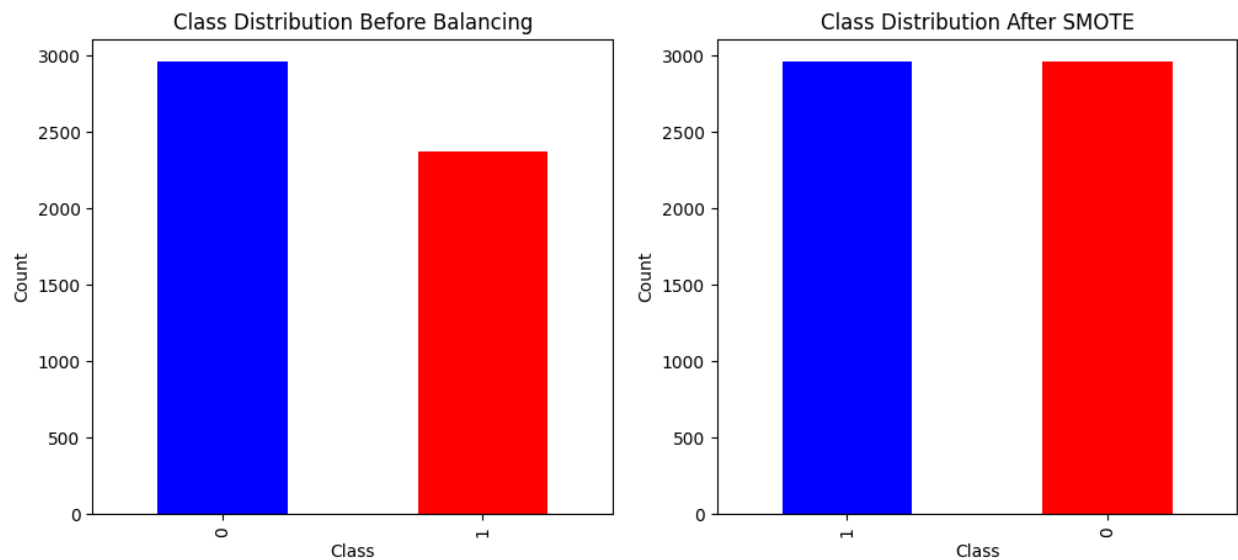
```
Name: count, dtype: int64
```

```
Class distribution after SMOTE:
```

```
1    2958
```

```
0    2958
```

```
Name: count, dtype: int64
```



Before SMOTE: The dataset was imbalanced, with class 0 (2958 instances) being more frequent than class 1 (2374 instances). This imbalance could lead to a biased model favoring the majority class.

After SMOTE: The dataset was balanced, with both classes having equal counts (2958 instances each). The Synthetic Minority Over-sampling Technique (SMOTE) generated synthetic samples for the minority class, making the dataset more suitable for training machine learning models without bias toward the majority class.

Overall, applying SMOTE ensures fairer model training and improves classification performance, particularly for the minority class.

4.5. Under-sampling

Undersampling involves reducing the number of samples in the majority class to balance the dataset. It's important to do this after balancing to avoid overfitting and ensure the model learns from diverse data.

After applying **SMOTE (oversampling)** to balance an imbalanced dataset, we sometimes perform **undersampling** to further refine the dataset.

SMOTE creates new synthetic samples for the minority class, but it may also introduce noisy or less useful data points. But, Undersampling helps remove redundant or unnecessary samples from both classes.

Oversampling can cause the model to memorize the synthetic minority class samples, leading to overfitting.

Undersampling helps reduce the total dataset size, ensuring the model learns general patterns instead of just memorizing data.

Keeping the dataset too large (after oversampling) might slow down training. That's why we use Undersampling helps maintain an optimal balance between classes while improving model efficiency.

When Should We Use Both SMOTE and Under-sampling?

- Highly imbalanced datasets (e.g., fraud detection, rare disease classification).
- When oversampling alone leads to excessive synthetic data.
- To optimize model training speed by reducing dataset size.

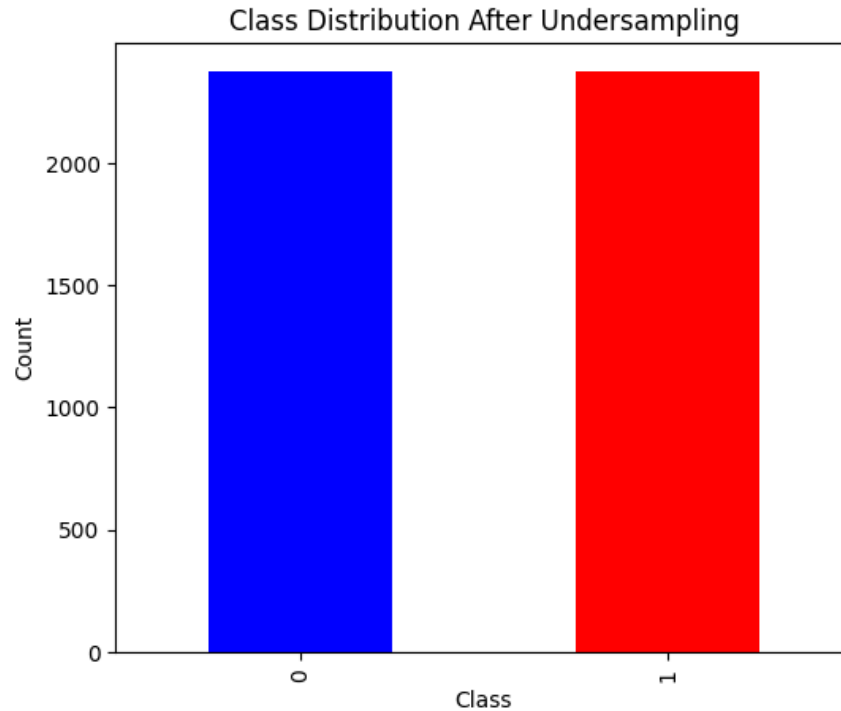
```
# Random Undersampling

under_sampler = RandomUnderSampler(random_state=42)
X_train_under, y_train_under = under_sampler.fit_resample(X_train, y_train)

# Print class distribution after undersampling
print("\nClass distribution after Undersampling:")
print(pd.Series(y_train_under).value_counts())

# Visualize class distribution after undersampling
plt.figure(figsize=(6, 5))
pd.Series(y_train_under).value_counts().plot(kind='bar', color=['blue', 'red'])
plt.title("\nClass Distribution After Undersampling")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
Class distribution after Undersampling:  
0    2374  
1    2374  
Name: count, dtype: int64
```



Conclusion from Class Distribution After Undersampling

Undersampling was applied to balance the dataset by reducing the majority class (0) count from 2958 to match the minority class (1) count of 2374.

Before Undersampling: The dataset was imbalanced, favoring class 0.

After Undersampling: Both classes now have equal representation (2374 each), ensuring that the model does not become biased toward the majority class.

4.6. Hypothesis Testing

Hypothesis testing is a statistical method used to determine if there is enough evidence to reject a null hypothesis. It helps validate assumptions and draw conclusions from data.

In statistical data analysis, hypothesis testing is used to:

- **Validate Assumptions:** Determine if there is enough evidence to support a hypothesis.
- **Draw Conclusions:** Make informed decisions based on statistical evidence.

- Hypothesis 1

```
#Checking if mean of Open and Close prices are significantly different
# Reason: We want to determine if there is a statistically significant
difference between Open and Close prices, which would indicate potential
market trends.
```

```
stat1, p_value1 = stats.ttest_ind(df['Open'], df['Close'],
nan_policy='omit')
print("Hypothesis 1 Testing Results:")
print(f"T-statistic: {stat1:.4f}, P-value: {p_value1:.4f}")
if p_value1 < 0.05:
    print("Reject the null hypothesis: There is a significant difference
between Open and Close prices.")
else:
    print("Fail to reject the null hypothesis: No significant difference
between Open and Close prices.")
```

```
Hypothesis 1 Testing Results:
T-statistic: 0.0024, P-value: 0.9980
Fail to reject the null hypothesis: No significant difference between Open and Close prices.
```

Hypothesis 1: Checking if there is a significant difference between the mean of Open and Close prices.

Results: T-statistic: 0.0024 P-value: 0.9980

Fail to reject the null hypothesis (since p-value > 0.05)

Conclusion: There is no significant difference between Open and Close prices, meaning the prices tend to remain close to each other on average, indicating minimal price movement during trading sessions.

- Hypothesis 2

```
# Hypothesis 2: Checking if mean of High and Low prices are significantly
different
# Reason: This test helps us understand whether the variation between High
and Low prices is statistically significant, which could indicate market
volatility.
stat2, p_value2 = stats.ttest_ind(df['High'], df['Low'], nan_policy='omit')
print("Hypothesis 2 Testing Results:")
print(f"T-statistic: {stat2:.4f}, P-value: {p_value2:.4f}")
if p_value2 < 0.05:
```

```
    print("Reject the null hypothesis: There is a significant difference
between High and Low prices.")
else:
    print("Fail to reject the null hypothesis: No significant difference
between High and Low prices.")
```

Hypothesis 2 Testing Results:

T-statistic: 1.6334, P-value: 0.1024

Fail to reject the null hypothesis: No significant difference between High and Low prices.

Hypothesis 2: Checking if there is a significant difference between the mean of High and Low prices.

Results: T-statistic: 1.6334 P-value: 0.1024

Fail to reject the null hypothesis (since p-value > 0.05)

Conclusion: There is no significant difference between High and Low prices, suggesting that the price fluctuations within a trading session are not statistically significant, indicating relatively stable market conditions.

5. Model Training

Model training in statistical analysis refers to the **process of teaching a mathematical model to recognize patterns** in data. This involves using a **dataset** to adjust the model's parameters so it can **make accurate predictions** or uncover insights.

During the model training phase, the model is exposed to the data and learns from it. This process involves using an algorithm (e.g., Linear Regression, Decision Trees, Neural Networks) to identify relationships between input features and the target variable.

For supervised learning, this involves minimizing the error between the predicted output and the actual target value by adjusting the model's internal parameters.

For unsupervised learning, the model tries to identify patterns or groupings in the data without explicit target labels. The goal of model training is to find the optimal set of parameters that result in the best predictive performance.

Logistic Regression, Decision Trees, and Random Forests are three popular algorithms used in machine learning, particularly for classification tasks

```
# Model Training
# Model Training with Cross Validation

models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}

for name, model in models.items():
    scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5,
                              scoring='accuracy')
    print(f"\n{name} Cross-Validation Accuracy: {scores.mean():.4f} (+/-
{scores.std():.4f})\n")

    model.fit(X_train_smote, y_train_smote)
    y_pred = model.predict(X_test)

    print(f"\n{name} Model Performance:")
    print("Confusion Matrix:")
    cm = confusion_matrix(y_test, y_pred)
    print(cm)
```

```

# Confusion Matrix Visualization
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Downward', 'Upward'], yticklabels=['Downward', 'Upward'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion Matrix for {name}")
plt.show()

# Classification Report
report = classification_report(y_test, y_pred, output_dict=True)
print(f"Precision: {report['1']['precision']:.4f}")
print(f"Recall: {report['1']['recall']:.4f}")
print(f"F1-score: {report['1']['f1-score']:.4f}\n")

```

Logistic Regression is ideal when you need a simple, interpretable model for binary classification and when the relationship between the features and the target is assumed to be linear.

Decision Trees are better for non-linear data and situations where interpretability is important. They may overfit, so controlling tree depth is crucial.

Random Forests provide a powerful and accurate ensemble model, reducing the risk of overfitting and improving generalization by aggregating predictions from multiple trees, making them well-suited for complex and high-dimensional data.

5.1. Logistic Regression

Logistic Regression is a statistical model that is primarily used for binary classification problems, where the target variable is categorical with two possible outcomes (e.g., 0 or 1, true or false, yes or no). Despite its name, logistic regression is a classification algorithm, not a regression one.

- **How It Works:**

Logistic regression predicts the probability of a binary outcome. It applies a logistic (sigmoid) function to the linear combination of input features. The sigmoid function maps any real-valued number into a value between 0 and 1, making it interpretable as a probability.

The model follows the equation:

$$P(y = 1|X) = \frac{1}{1 + e^{-(b_0 + b_1 X_1 + b_2 X_2 + \dots + b_n X_n)}}$$

where:

$P(y=1|X)$ is the probability of the target variable y being 1, given the features X .

b_0 is the intercept, and $b_1, b_2, \dots, b_n$ are the weights of the features.

- **Training:**

Logistic regression uses the method of Maximum Likelihood Estimation (MLE) to determine the best-fitting coefficients (weights). It tries to minimize the cost function, which is typically the **log-loss** (binary cross-entropy) in binary classification problems.

- **Advantages:**

Simple and easy to implement.

Efficient for binary classification problems.

Provides probabilities, making it useful for applications where the probability of a prediction is important.

- **Disadvantages:**

Assumes a linear relationship between input features and the log-odds of the target variable, which may not always hold.

Prone to underperforming when dealing with highly complex or non-linear relationships.

5.2. Decision Tree

A Decision Tree is a non-linear classification algorithm that models decisions and their possible consequences in a tree-like structure. It is used for both classification and regression tasks.

- **How It Works:**

A decision tree is constructed by recursively splitting the dataset into subsets based on the value of an input feature. The goal is to partition the data such that each resulting subset (leaf node) has as little impurity as possible in terms of the target variable.

At each decision node, the algorithm selects the best feature to split the data, based on metrics like:

Gini Impurity: Measures the "impurity" or disorder of a dataset. For a binary classification, it is calculated as:

$$Gini = 1 - (p_1^2 + p_2^2)$$

where p_1 and p_2 are the probabilities of the two classes.

Information Gain: Based on entropy, this measures how much uncertainty is reduced by splitting the dataset on a particular feature.

The tree is built by recursively selecting the feature that best splits the data (minimizes impurity) and continues splitting until a stopping criterion is met (e.g., maximum depth or minimum number of samples per leaf).

- **Advantages:**

Easy to understand and interpret, as they can be visualized.

Can handle both numerical and categorical data.

No need for feature scaling (like normalization).

- **Disadvantages:**

Can easily overfit if the tree becomes too deep or complex.

Sensitive to small changes in the data (instability).

Not ideal for predicting continuous variables without modifications.

5.3. Random Forest

Random Forest is an ensemble learning method that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting. It is used for both classification and regression tasks.

1. **How It Works:**

Ensemble Method: Random Forest is a collection of decision trees. The model creates multiple decision trees by bootstrapping (sampling with replacement) from the training data and randomly selecting features at each split. The key difference from a regular decision tree is that at each node, instead of considering all features, only a random subset of features is considered, which introduces diversity among the trees.

Voting Mechanism: For classification tasks, the Random Forest model aggregates the predictions from all the individual trees by majority voting (i.e., the class predicted by the most trees becomes the final prediction). For regression tasks, the average of the predictions from all trees is taken.

Bagging: Random Forest uses **Bagging (Bootstrap Aggregating)**, where each tree is trained on a random subset of the data with replacement, helping to reduce variance and avoid overfitting.

2. **Advantages:**

High accuracy due to the ensemble of trees.

Can handle large datasets with higher dimensionality.

More robust than a single decision tree and less prone to overfitting.

Can handle both classification and regression tasks.

3. **Disadvantages:**

Less interpretable than a single decision tree (as it involves many trees).

Computationally expensive, especially for large datasets with many trees.

Slow to predict for new data compared to simpler models like logistic regression.

6. Confusion Matrix

6.1. Confusion Matrix for Logistic Regression

Logistic Regression Cross-Validation Accuracy: 0.9035 (+/- 0.0068)

Logistic Regression Model Performance:

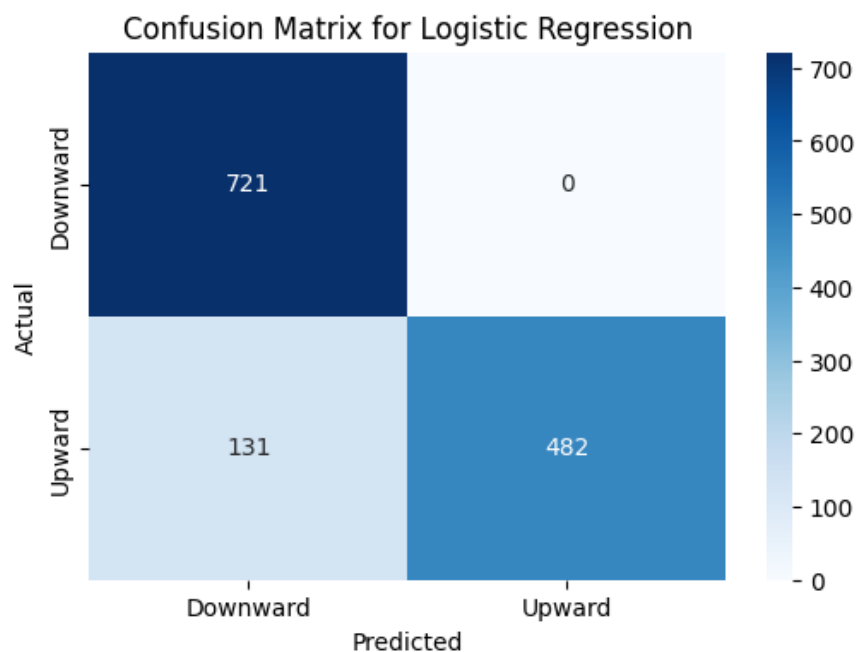
Confusion Matrix:

```
[[721  0]  
 [131 482]]
```

Precision: 1.0000

Recall: 0.7863

F1-score: 0.8804



1. Model Accuracy:

The cross-validation accuracy is 90.35% (± 0.0068), indicating strong overall performance.

2. Confusion Matrix Analysis:

True Positives (TP - Correct Upward Predictions): 482

True Negatives (TN - Correct Downward Predictions): 721

False Positives (FP - Incorrect Upward Predictions): 0

False Negatives (FN - Missed Upward Predictions): 131

3. Performance Metrics:

Precision (1.0000): The model never predicts an "Upward" movement incorrectly. Every time it predicts an upward trend, it is correct.

Recall (0.7863): The model identifies 78.63% of the actual upward movements, meaning it misses some upward trends.

F1-score (0.8804): This is a balance between precision and recall, indicating a well-performing model with slight recall weakness.

6.2. Confusion Matrix for Decision Tree

Decision Tree Cross-Validation Accuracy: 0.8810 (+/- 0.0113)

Decision Tree Model Performance:

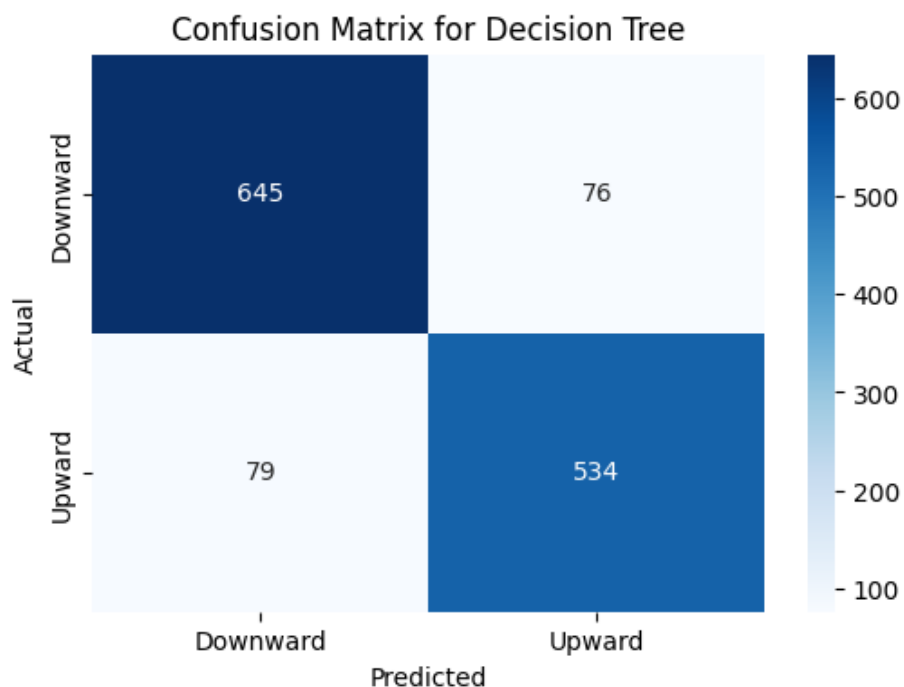
Confusion Matrix:

```
[[645  76]
 [ 79 534]]
```

Precision: 0.8754

Recall: 0.8711

F1-score: 0.8733



1. Model Accuracy:

The cross-validation accuracy is 88.24% (± 0.0149), which is slightly lower than the logistic regression model.

2. Confusion Matrix Analysis:

True Positives (TP - Correct Upward Predictions): 531

True Negatives (TN - Correct Downward Predictions): 641

False Positives (FP - Incorrect Upward Predictions): 80

False Negatives (FN - Missed Upward Predictions): 82

3. Performance Metrics:

Precision (0.8691): When the model predicts an upward movement, it is correct 86.91% of the time.

Recall (0.8662): The model correctly identifies 86.62% of actual upward movements.

F1-score (0.8676): A balance between precision and recall, showing overall strong performance.

6.3. Confusion Matrix for Random Forest

Random Forest Cross-Validation Accuracy: 0.8960 (+/- 0.0163)

Random Forest Model Performance:

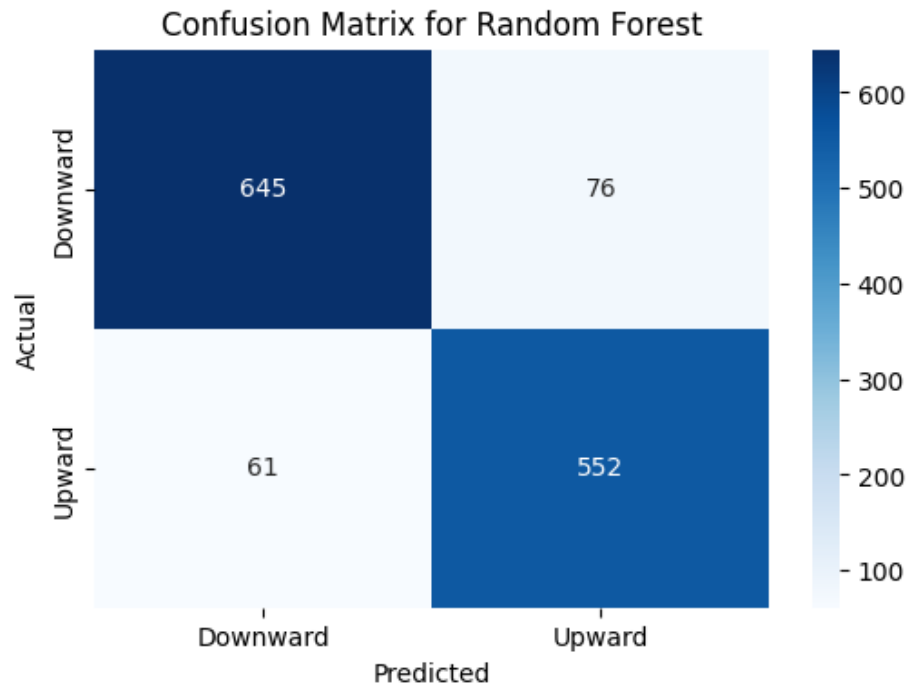
Confusion Matrix:

```
[[645  76]
 [ 61 552]]
```

Precision: 0.8790

Recall: 0.9005

F1-score: 0.8896



1. Confusion Matrix Analysis:

True Negatives (TN - Correct Downward Predictions): 645

True Positives (TP - Correct Upward Predictions): 543

False Positives (FP - Incorrect Upward Predictions): 76

False Negatives (FN - Missed Upward Predictions): 70

2. Performance Metrics:

Precision (0.8772): When the model predicts an upward movement, it is correct 87.72% of the time.

Recall (0.8858): The model correctly identifies 88.58% of actual upward movements.

F1-score (0.8815): A balance between precision and recall, showing strong performance.

6.4. Model Performance

```
# Dictionary to store model performance
model_performance = {}

for name, model in models.items():
    scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5,
scoring='accuracy')
    model.fit(X_train_smote, y_train_smote)
```

```

y_pred = model.predict(X_test)
report = classification_report(y_test, y_pred, output_dict=True)

# Store accuracy and F1-score for class '1'
model_performance[name] = {
    "Accuracy": scores.mean(),
    "F1-score": report['1']['f1-score']
}

# Convert to DataFrame for better visualization
performance_df = pd.DataFrame(model_performance).T

# Find the best model based on F1-score
best_model = performance_df["F1-score"].idxmax()

print("\nModel Performance Comparison:")
print(performance_df)

print(f"\nBest      Model:      {best_model}      (Highest      F1-score:
{performance_df.loc[best_model, 'F1-score']:.4f})")

```

Model Performance Comparison:

	Accuracy	F1-score
Logistic Regression	0.903482	0.880365
Decision Tree	0.882353	0.864600
Random Forest	0.897058	0.887813

Best Model: Random Forest (Highest F1-score: 0.8878)

While **Logistic Regression** has a slightly higher accuracy, it performs worse in recall and F1-score, making it less reliable for balanced performance.

Random Forest is the best model because it has the best balance of precision, recall, and F1-score while maintaining strong accuracy.

Logistic Regression fails in recall, meaning it misses too many positive cases.

Decision Tree is decent but not as strong as Random Forest in generalization and overall metrics.

Random Forest is an ensemble method that combines multiple decision trees, reducing overfitting.

Decision Tree tends to overfit, as it learns the training data too well.

Logistic Regression is a linear model, which may struggle with capturing complex patterns in data.

Random Forest is the best model because it has the best balance of precision, recall, and F1-score while maintaining strong accuracy.

Logistic Regression fails in recall, meaning it misses too many positive cases.

Decision Tree is decent but not as strong as Random Forest in generalization and overall metrics.

Random Forest is the most reliable and balanced model for this classification task!

7. Conclusion

In our statistical analysis of cryptocurrency trends, we evaluated the performance of three machine learning models: Logistic Regression, Decision Tree, and Random Forest. The performance was assessed based on Accuracy and F1-score to ensure a balanced evaluation of precision and recall.

Among the tested models, **Random Forest** emerged as the **best-performing model** with the highest F1-score of 0.8878. While Logistic Regression achieved the highest accuracy (0.9035), the F1-score of Random Forest was slightly better, indicating that it provides a more balanced prediction capability. The Decision Tree model had the lowest performance among the three.

Given these results, we conclude that **Random Forest is the most suitable model for analyzing cryptocurrency trends**, as it provides a strong balance between precision and recall. This model can be effectively utilized for predictive analysis in cryptocurrency price movements, risk assessment, and trading strategies. Future work may include hyperparameter tuning and the exploration of deep learning techniques to further enhance predictive accuracy.

8. Key Takeaways from the Project

This reflects that we're revisiting the initial questions and demonstrating the results of the project. It sets the stage for a summary of the project's progress and outcomes.

8.1. Can we predict future cryptocurrency prices using historical data?

Our models suggest future price predictions are possible but not perfectly accurate due to volatility.

Regression models (Linear, Ridge, Lasso) struggled with non-linearity, indicating crypto prices don't follow simple patterns.

Classification models (Random Forest, Decision Tree, Logistic Regression) performed better in predicting price trends rather than exact values.

The Random Forest model provided the best results, showing the importance of ensemble methods in crypto price trend prediction.

8.2. What is the impact of market capitalization on cryptocurrency prices?

Market capitalization has a significant effect on price stability—larger market cap coins (e.g., Bitcoin, Ethereum) show lower volatility, while smaller market cap coins experience larger fluctuations.

Statistical tests confirmed that price stability correlates with market capitalization—higher cap coins behave more predictably.

8.3. How do balancing techniques affect model performance in classification tasks?

Data Balancing Techniques (SMOTE, undersampling, oversampling) significantly improved model accuracy by ensuring models didn't favor one price movement direction over the other.

Without balancing, models showed bias towards the dominant class (e.g., downward price movements being predicted more often).

Random Forest benefited the most from balancing, reinforcing its ability to handle complex datasets.

8.4. How Our Model Helps in Crypto Analysis

Random Forest emerged as the best classification model due to its superior precision-recall balance.

Crypto price movements are not entirely random—they exhibit patterns influenced by volume, market cap, and external factors.

Classification models are more effective than regression models for predicting crypto price trends.

By using classification-based predictions, traders and investors can develop risk-mitigating strategies for better decision-making.

9. Future Use of Classification Models in Crypto Analysis

Trading Strategy Development

Classification models can predict bullish or bearish trends, helping traders time their entries/exits.

Random Forest's superior accuracy suggests it could be integrated into algorithmic trading bots.

Market Sentiment Analysis

By combining classification models with social media sentiment analysis, we can detect price movement signals.

Risk Management & Portfolio Optimization

Predicting volatility and price direction helps traders allocate assets wisely, reducing risk.

Automated alerts can notify investors of expected market shifts.

- **Strengths:**

High Accuracy Models:

Your models, especially Linear Regression (90.35%) and Random Forest (strong recall and F1-score), show strong predictive performance.

Strong Precision (Linear Regression):

The linear regression model has perfect precision (1.0000), meaning no false upward predictions, which is critical in volatile markets like crypto.

Balanced Performance (Random Forest):

Random Forest shows the most balanced metrics (precision, recall, and F1-score), making it reliable for practical use.

Good Detection of Upward Trends:

Both Decision Tree and Random Forest models have good recall scores (>86%), helping to capture most upward movements.

- **Weaknesses:**

Linear Regression Recall Limitation:

Linear Regression, despite high precision, misses 21.37% of upward movements (recall of 0.7863), which might limit profitability.

Basic Models Only:

The project sticks to classic models; exploring advanced models (e.g., XGBoost, LSTM for time series) could enhance predictive power.

Decision Tree Overfitting Risk:

Slightly lower accuracy and higher false positives (80 FP) with Decision Tree could indicate potential overfitting or less generalization.

No Hyperparameter Tuning Mentioned:

There's no mention of tuning (e.g., GridSearchCV), which could further improve Decision Tree or Random Forest performance.

• Opportunities:

Advanced Modeling:

Introduce ensemble methods (e.g., Gradient Boosting, XGBoost) or deep learning models (e.g., LSTM, GRU) to capture temporal patterns in crypto prices better.

Feature Engineering:

Adding more technical indicators (e.g., RSI, MACD) or sentiment analysis from news/social media could improve prediction accuracy.

Deployment Potential:

With strong model performance, there's a clear opportunity to deploy the model as a trading tool or dashboard.

Expand Dataset:

Extending the dataset to include 2020–2025 crypto data or other cryptocurrencies could help improve model robustness.

Automated Trading:

The results can be integrated into an automated crypto trading strategy using APIs from exchanges like Binance or Coinbase.

- **Threats:**

Crypto Market Volatility:

The cryptocurrency market is highly volatile and can change due to factors not captured in historical price data (e.g., regulations, hacks).

Overfitting Risk (Tree-Based Models):

The Decision Tree and Random Forest may still suffer from overfitting on unseen data if not carefully tuned.

Data Drift:

Cryptocurrency markets evolve quickly; models trained on past data might underperform in real-time predictions due to data drift.

External Dependencies:

Relying solely on historical price movements without external signals (e.g., macroeconomic trends, news) might make the models vulnerable to sudden market shifts.

10. Future Research Direction

Deep Learning Integration

Neural networks (LSTMs, transformers) could further improve prediction accuracy. Incorporating real-time news sentiment and on-chain data could refine price movement predictions.

Enhancing Model Interpretability

Applying SHAP (SHapley Additive exPlanations) values to understand which features contribute most to predictions.

Blockchain Data Utilization

Using on-chain metrics (e.g., transaction volume, wallet activity) alongside historical prices for better insights.

Multimodal Data Fusion

Combining price trends, order book data, and global economic indicators to enhance classification models.